



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A PROPOSAL DRAFT
ON
HUMAN PRESENCE DETECTION USING DISTRIBUTED INFERENCE ON
RESOURCE CONSTRAINED EDGE DEVICES**

Submitted By:

Pratik Khatiwada (THA079BEI025)
Preeti Rijal (THA079BEI028)
Sampada Ghimire (THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

June , 2026

ABSTRACT

The field of monitoring and surveillance requires the use of high resolution image sensors but the computational power required to process such data is very high and becomes a significant challenge for edge device processing. In order to process the high resolution image in an edge device requires an aggressive spatial downsampling which destroys the fine details necessary for reliable detection of small and distant subjects. This work presents a distributed inference architecture for real-time Human Presence Detection (HPD) on microcontroller based systems. Rather than downsampling the full frame, the pipeline partitions each captured image into a grid of smaller tiles, allowing each tile to be inferred independently at full local resolution. The resulting tile-level predictions are passed to a lightweight aggregator, which combines outputs across all tiles to produce a single classification score. End-to-end benchmarking reveals that frame capturing and tiling completes in 4 ms, aggregator classification in 0.342 ms, and takes 472 ms for feature transmission per tile. The dominant bottleneck is tile level inference on the ESP32-S3, which requires 2,657 ms per tile. The total critical-path latency comes to 18.9 seconds approximately against the 1,000 ms target. The results confirm architectural correctness across all pipeline stages while clearly identifying memory bound inference latency and sequential tile fragmentation as the primary targets for optimisation in future work.

Keywords: *Human presence detection, distributed tile-based inference, local binary convolutional neural network, attention mechanism, TinyML, post-training quantization, ESP32, edge computing.*

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	iii
LIST OF TABLES	iv
LIST OF ABBREVIATIONS	iv
1. Introduction	1
1.1. Background	1
1.2. Motivation	1
1.3. Problem Statement	2
1.4. Objectives	2
1.5. Project Scope	2
1.6. Problem Applications	3
2. LITERATURE REVIEW	4
2.1. Related Works	4
2.2. Existing Systems and Limitations	6
3. PROPOSED SYSTEM/METHODOLOGY	7
3.1. Model Selection and Optimization Pipeline	7
3.2. Hardware Requirements and system configuration	14
3.3. Hardware Used	19
3.4. Evaluation Metrics and Performance Framework	22
4. REQUIREMENTS	24
4.1. Hardware Requirements	24
4.2. Software & Framework Requirements	24
4.3. Non-functional Requirements	24
5. EXPECTED OUTCOMES	26
A. APPENDICES	27
A.1. PROJECT BUDGET	27
A.2. PROJECT TIMELINE	27
REFERENCES	28

LIST OF FIGURES

Figure 3-1. Multi-Tile Attention-Based CNN Architecture	8
Figure 3-2. Detailed Schematic of the Local Binary Convolution Module	9
Figure 3-3. System Architecture showing main components and their interactions.	15
Figure 3-4. Detailed Data Flow Diagram of the Tiling and Inference Pipeline. . .	16
Figure 3-5. Wiring Interface between Programming (FTDI), Processing (ESP32-S3), and Visualization (I2C Liquid Crystal Display (LCD)) Units.	19

LIST OF TABLES

Table 2-1. Comparison of Related Works with Proposed System	6
Table 3-1. Wiring the FTDI to ESP32-CAM	20
Table 3-2. Wiring the I2C LCD to ESP32	21
Table A-1. Budget Estimation	27
Table A-2. Gantt Chart for Project Timeline	27

1. INTRODUCTION

1.1. Background

The ability to capture, interpret, and replicate human motion has long been a fundamental challenge in robotics, computer vision, and human-computer interaction. Traditional motion capture systems rely on expensive marker-based setups, wearable inertial measurement units (IMUs), or specialized depth sensors such as the Microsoft Kinect and Intel RealSense, all of which impose significant hardware constraints and deployment costs. Recent advances in deep learning-based pose estimation, particularly lightweight frameworks such as MediaPipe developed by Google, have enabled accurate real-time extraction of human joint coordinates from standard RGB video without specialized hardware.

Simultaneously, physics-based simulation platforms such as PyBullet have matured into robust environments for replicating articulated body motion, providing physically accurate rendering of joint movement and a foundation for generating structured motion datasets. The convergence of these two technologies — vision-based pose estimation and physics-based simulation — presents an opportunity to build affordable, accessible motion capture systems that require only standard cameras.

In the context of industrial robotics and assistive automation, large-scale datasets of human arm movement are essential for training robotic manipulators to perform tasks mimicking natural human motion. However, the cost and complexity of existing data collection pipelines remain a barrier to widespread dataset generation. A vision-based, multi-camera system capable of capturing full arm-to-fingertip motion in real-time and replicating it in simulation addresses this barrier directly.

1.2. Motivation

Robotic arms capable of replicating human arm motion have significant potential in industrial automation, assistive robotics, teleoperation, and rehabilitation engineering. A core requirement for training such systems is access to large, diverse datasets of human arm trajectories annotated with joint coordinates and angles. Existing approaches to generating such datasets either rely on expensive hardware (RGB-D sensors, IMU suits) or produce offline, non-real-time outputs that are difficult to scale.

The motivation for MotionBridge arises from the absence of a low-cost, real-time, vision-only pipeline that combines multi-camera capture, neural network-based pose estimation, three-dimensional reconstruction, and physics simulation into a single integrated system. By using only standard USB webcams and open-source software, the proposed system can be deployed in resource-constrained environments such as

academic laboratories, small research groups, and developing-world institutions where expensive motion capture infrastructure is unavailable.

Furthermore, no existing reviewed work captures both full arm joint trajectories and detailed finger joint coordinates (21 points per hand) from standard RGB cameras and simultaneously replicates arm movement in simulation. MotionBridge addresses this gap, producing a richer dataset than currently available vision-based approaches.

1.3. Problem Statement

Existing human arm motion capture systems for robotic dataset generation suffer from one or more of the following limitations: reliance on expensive depth-sensing hardware, dependence on wearable sensors, absence of real-time simulation replication, operation on monocular camera inputs that cannot resolve depth accurately, or omission of finger-level joint data. These constraints prevent scalable and affordable motion dataset generation for robotic arm training.

This project addresses the problem of building a low-cost, real-time system that captures three-dimensional human arm and finger joint motion using only standard RGB cameras, replicates the captured motion in a physics-based simulation environment, and produces a structured dataset of joint coordinates and angles for downstream robotic arm training — without requiring specialized depth sensors, wearable hardware, or GPU-accelerated inference.

1.4. Objectives

1.5. Project Scope

1.5.1. Capabilities

- Real-time capture of human arm movement (shoulder, elbow, wrist) using three standard USB webcams.
- Detection of 21 finger joint landmarks per hand using MediaPipe Hands in addition to full arm keypoints.
- Three-dimensional joint coordinate reconstruction via confidence-weighted triangulation using calibrated multi-camera geometry.
- Real-time replication of arm movement in PyBullet physics simulation using Inverse Kinematics.
- Generation of a structured dataset containing per-frame 3D joint coordinates, joint angles, and timestamps.
- Occlusion robustness through multi-camera redundancy when one camera’s view is blocked, remaining cameras compensate.

1.5.2. Limitations

- Finger joint movement is captured and recorded in the dataset but is not replicated in PyBullet simulation, as standard arm URDF models do not include articulated finger structures.
- The system is limited to single-person capture; multi-person simultaneous tracking is outside scope.
- Accuracy depends on initial camera calibration quality; dynamic camera repositioning requires recalibration.
- The system does not include robot arm training or policy learning; it generates the dataset only.
- Performance may degrade under extreme lighting conditions or significant self-occlusion not resolvable by three cameras.
- The system does not support full-body motion capture only the arm from shoulder to fingertip.

1.6. Problem Applications

The dataset and simulation output produced by MotionBridge have direct applications in the following domains:

- **Industrial Automation:** Providing training data for robotic arms performing labour-intensive factory tasks such as assembly, sorting, and load-carrying that mimic human arm motion.
- **Assistive Robotics:** Generating motion trajectories for robotic prosthetics and assistive devices that replicate natural human arm and hand movement.
- **Teleoperation:** Serving as a reference system for mapping human operator arm motion to remote robotic manipulators.
- **Rehabilitation Engineering:** Capturing and analysing patient arm movement for physiotherapy assessment and progress monitoring.
- **Imitation Learning Research:** Supplying labelled demonstration datasets for behavioral cloning and inverse reinforcement learning pipelines.
- **Simulation-Based Training:** Using the PyBullet output as a reference motion environment for reinforcement learning agents.

2. LITERATURE REVIEW

2.1. Related Works

Human Pose Estimation (HPE) forms the foundational stage of any vision-based motion capture system. Kim et al. [1] demonstrated the effectiveness of MediaPipe Pose in a two-stage pipeline running on a single-board computer mounted on a mobile robot, achieving a mean joint coordinate difference of 0.097 m and an average angle difference of 10.017 degrees per joint across six daily poses. The study confirmed that MediaPipe provides a practical balance between computational cost and accuracy for real-time embedded applications, directly supporting its adoption as the pose estimation backbone in the proposed system.

Sarkar et al. [2] implemented a multi-camera system for close-proximity human-robot collaboration in construction environments, using MediaPipe Pose to extract 15 keypoints per frame from each camera view and converting them to 3D coordinates through rotation, translation, and homogeneous transformation matrices derived from known camera positions and orientations. This work is the closest architectural match to the proposed pipeline, validating the feasibility of combining MediaPipe with a multi-camera setup for real-world 3D joint reconstruction without specialized depth hardware.

Zell et al. [3] presented a multimodal human motion dataset containing 51,051 poses from 71 persons, obtained by extracting 2D keypoints using MediaPipe Human Pose Landmarker from virtual multi-camera images and computing corresponding 3D keypoints by linear triangulation. Their dataset was explicitly intended for transforming computer vision pose solutions into biomechanical simulations, constituting a direct proof of concept for the MediaPipe-plus-triangulation pipeline proposed in MotionBridge.

Bultmann and Behnke [4] proposed a distributed multi-camera architecture where 2D joint detection is performed locally at each smart edge sensor and only the semantic skeleton representation is transmitted to a central backend for 3D triangulation. Their system achieves real-time 3D pose estimation while significantly reducing network bandwidth, and incorporates prior human skeleton knowledge into the triangulation backend to improve robustness — an architectural approach directly relevant to the distributed and real-time requirements of the proposed system.

Bragagnolo et al. [5] addressed the known limitation of standard triangulation under occlusion by performing multi-view fusion on 3D skeletons from monocular methods rather than fusing noisy 2D features, applying reprojection error optimization with limb-length symmetry constraints. Their method significantly outperformed standard

triangulation in human-robot collaboration scenarios with heavy occlusion, identifying a clear accuracy upgrade path for the proposed system beyond its baseline triangulation implementation.

Leroy et al. [6] extended multi-view 3D pose estimation to uncalibrated camera setups through Probabilistic Triangulation, using Monte Carlo sampling to iteratively estimate camera pose distributions. Their experiments on Human3.6M and CMU Panoptic benchmarks demonstrated that accurate 3D reconstruction is achievable without fixed camera calibration, which is relevant when the proposed system is deployed in environments where precise camera recalibration is impractical.

Liang et al. [7] proposed the most directly related simulation-based work, integrating 3D HPE with PyBullet for robotic arm motion imitation. Their pipeline extracts human arm motion from video using a Strided Transformer network, retargets the human morphology to a robot URDF model, generates reference trajectories in PyBullet using its built-in Inverse Kinematics solver, and applies deep reinforcement learning for policy training. However, their system operates offline on monocular pre-recorded video and targets robot policy training rather than dataset generation, distinguishing it from the real-time multi-camera focus of MotionBridge.

Jiang et al. [8] demonstrated real-time multi-person arm motion imitation using a single RGB-D camera and an improved retargeting algorithm, achieving consistent arm configuration and precise end-effector tracking for multi-robot collaboration tasks. While their system confirms that real-time arm motion replication is feasible, it relies on expensive RGB-D hardware for depth acquisition, whereas MotionBridge targets equivalent real-time performance using only standard RGB cameras with computational triangulation, substantially lowering the deployment cost.

Faria et al. [9] presented a teleoperation framework combining MediaPipe landmark detection with an Intel RealSense RGB-D camera for 3D joint reconstruction, mapping the results to a robot arm’s kinematic constraints in real-time. Their work explicitly noted that vision-only estimation remains limited for axial rotations such as elbow and wrist yaw under occlusions — a limitation that the multi-camera triangulation approach in MotionBridge is specifically designed to mitigate by providing redundant viewpoints.

Moya-Alcover et al. [10] presented seven motion capture datasets of professional industrial gestures performed by skilled craftsmen and factory operators, recorded using wearable inertial sensors. Their work is motivated by the same industrial and labour application domain described in MotionBridge — capturing arm motion for tasks involving factory work and load-carrying — but relies on instrumented suits rather than

vision-based capture, introducing hardware constraints that the proposed camera-based system avoids entirely.

2.2. Existing Systems and Limitations

A comparative analysis of the reviewed works reveals consistent gaps across three dimensions: hardware cost, depth resolution method, and system completeness. Table 2-1 summarises the key attributes of existing systems against the proposed MotionBridge system.

Table 2-1. Comparison of Related Works with Proposed System

Work	Year	Camera Setup	Depth Method	Simulation	Primary Goal
Liang et al.	2024	Monocular	Transformer	PyBullet	DRL Training
Jiang et al.	2025	Single RGB-D	Hardware	Real Robot	Robot Control
Faria et al.	2025	Single RGB-D	Hardware	Real Robot	Teleoperation
Sarkar et al.	2022	Multi RGB	Homogeneous	None	HRC Safety
Bragagnolo et al.	2024	Multi RGB	Multi-view Fusion	None	Pose Accuracy
Bultmann & Behnke	2021	Multi (edge)	Triangulation	None	Real-time 3D
Moya-Alcover et al.	2023	IMU-based	Inertial	None	Dataset
Zell et al.	2024	Virtual Multi	Triangulation	None	Dataset
MotionBridge	2025	Multi RGB	Conf.-Weighted Tri.	PyBullet	Simulation + Dataset

Systems that achieve real-time arm replication (Jiang et al., Faria et al.) depend on hardware depth sensors costing \$300 or more. Systems using standard cameras and triangulation (Sarkar et al., Zell et al., Bultmann and Behnke) omit the simulation replication stage entirely. The only system using PyBullet for arm motion replication (Liang et al.) operates offline on monocular input and produces no structured dataset. No reviewed work captures finger-level joint data alongside arm joints from standard RGB cameras. MotionBridge uniquely combines all four capabilities — multi-camera RGB capture, real-time pose estimation, confidence-weighted triangulation, PyBullet simulation replication, and full arm-to-fingertip dataset generation — in a single low-cost pipeline.

3. PROPOSED SYSTEM/METHODOLOGY

3.1. Model Selection and Optimization Pipeline

3.1.1. MobileNetV2 Architecture

The proposed system leverages a Convolutional Neural Network (CNN) based on a mobilenetv2 backbone, specifically optimized for binary classification tasks where $f_{\theta} : \mathbb{R}^{N \times 1 \times H \times W} \rightarrow \mathbb{R}^{N \times 2}$. The architecture is designed to balance high level feature extraction with the strict hardware constraints of Microcontroller Units (MCUs).

Grayscale Adaptation and Transfer Learning

To minimize memory consumption and computational latency, the model is adapted to process single channel grayscale images. Since the pretrained weights were originally optimized for rgb data, the first convolution layer's weights ($W \in \mathbb{R}^{C_{out} \times 3 \times k \times k}$) are averaged across the channel dimension:

$$Y = 0.299R + 0.587G + 0.114B \quad (3-1)$$

This technique retains low level feature detection, such as edge detectors and textures, while allowing the model to function on simplified Static Random Access Memory Static Random-Access Memory (SRAM) efficient inputs.

Classifier Head and Loss Function

The final classification is performed by a narrow bottleneck head ($1280 \rightarrow 64 \rightarrow 2$) using ReLU activation and dropout (0.2 and 0.1) to prevent overfitting. The resulting logits $\hat{y}$ are evaluated using Cross-Entropy Loss:

$$L = - \sum_{i=1}^2 y_i \log \left(\frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}} \right) \quad (3-2)$$

3.1.2. Local Binary Convolution Based Architecture

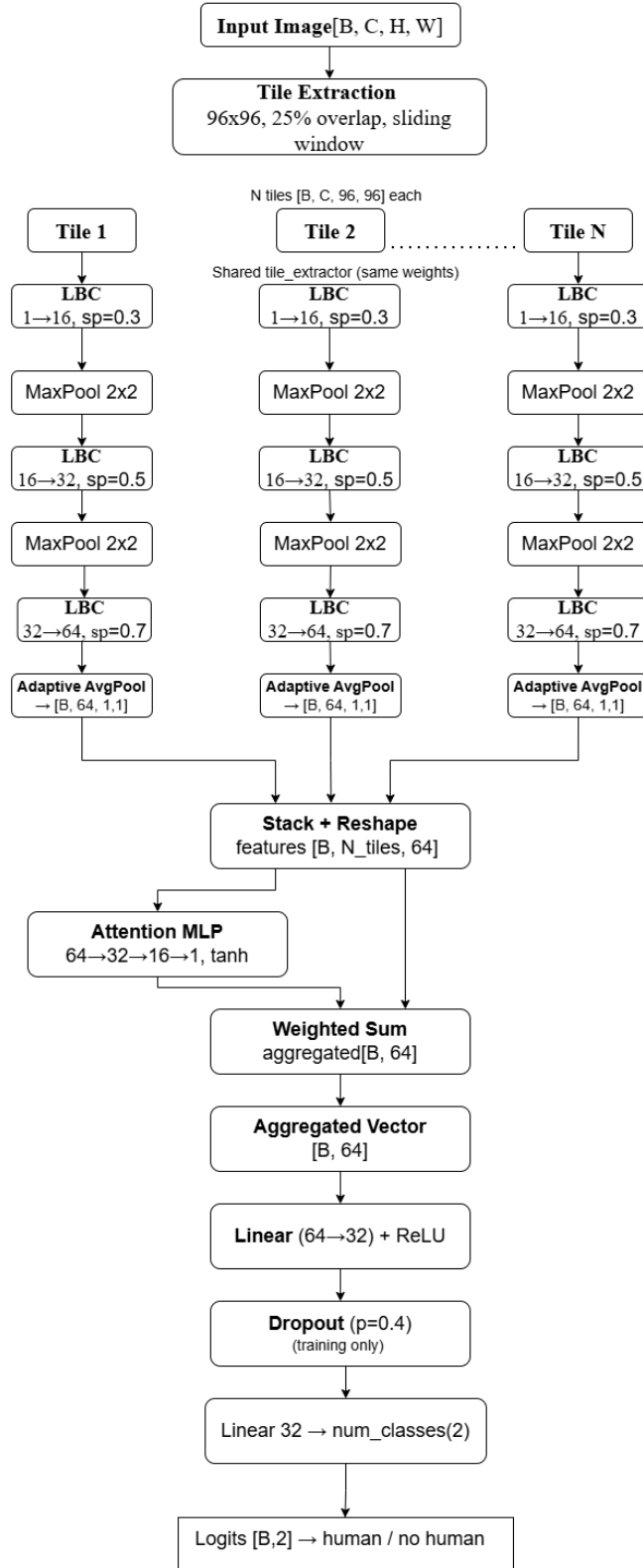


Figure 3-1. Multi-Tile Attention-Based CNN Architecture

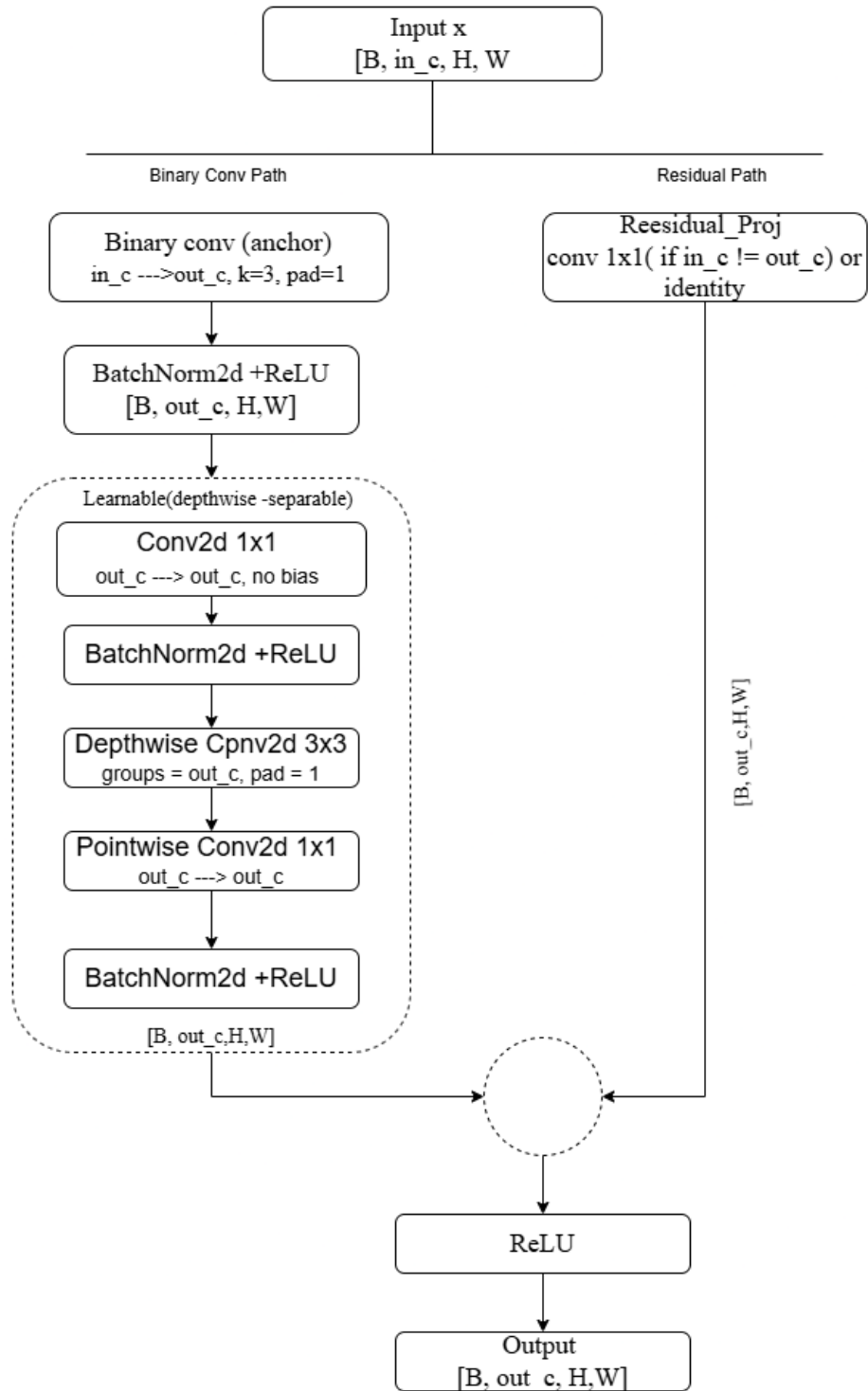


Figure 3-2. Detailed Schematic of the Local Binary Convolution Module

LBCLayer Overview

The LBCLayer is a hybrid convolutional layer designed to combine fixed binary convolutions (anchors) for efficient sparse structural feature capture with a learnable 1×1

convolution for task-specific adaptation. It is particularly suitable for resource-efficient networks and sparse feature representations, inspired by research on Local Binary Convolutions (LBC) and sparse binary kernels.

Mathematically, for an input $x \in \mathbb{R}^{N \times C_{in} \times H \times W}$, the output is defined as:

$$x_{out} = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x))) \quad (3-3)$$

Where:

- $\text{Conv}_{\text{binary}}$ uses fixed ternary weights $\in \{-1, 0, 1\}$.
- Conv1x1 is trainable and combines channels efficiently.

LBCLayer Components

1. Fixed Anchor Convolution

The purpose of this component is to extract general features without learning, thereby reducing the parameter count.

- **Weight generation:** The first step is the creation of weights matrix $W \sim \mathcal{N}(0, 1)$ and its sparsifying with parameter s :

$$\text{binary_weight}_{i,j,k,l} = \begin{cases} 1 & \text{if } W_{i,j,k,l} > \text{threshold} \\ -1 & \text{if } W_{i,j,k,l} < -\text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

- The sparsity parameter s will define how many zeros will there be in the matrix.
- This matrix of weights is saved as a buffer (`register_buffer`), which means it won't be trained afterwards.

The main idea is to capture general directional/edge patterns efficiently similar to handcrafted filters like Sobel, but stochastically generated.

2. Non-linearity (ReLU)

After the fixed binary convolution, non linearity is applied by ReLU: $x' = \text{ReLU}(x)$. Non linearity is introduced to help approximate complex functions, which also maintain sparsity and improving computational efficiency since there will be a lot of zeros.

3. Learnable 1x1 Convolution

The 1×1 kernel combines sparse feature maps into task relevant channels while keeping computational costs low.

$$x_{\text{final}} = W_{1 \times 1} * x' \quad (3-4)$$

Only this part is trainable, allowing the layer to adapt to specific tasks like human detection.

Forward Pass Flow

For a single LBCLayer, the flow is as follows:

1. The input to the layer is $x \in \mathbb{R}^{N \times C_{\text{in}} \times H \times W}$.
2. Perform binary convolution using fixed parameters: $x_1 = \text{Conv}_{\text{binary}}(x)$.
3. Use ReLU activation function: $x_2 = \text{ReLU}(x_1)$.
4. Perform 1×1 convolution using learnable parameters: $x_{\text{out}} = \text{Conv}_{1 \times 1}(x_2)$.
5. Output: $x_{\text{out}} \in \mathbb{R}^{N \times C_{\text{out}} \times H \times W}$.

Advantages of LBCLayer

- **Computational efficiency:** Fixed sparse weights reduce multiply adds, and 1×1 learnable convolutions are computationally cheap.
- **Stability:** Binary anchors are not updated, which avoids noisy gradients and the network only optimizes the final 1×1 convolution.
- **Sparsity Control:** The sparsity parameter allows control over network complexity and feature density so that encouraging sparse representations that improve generalization.
- **Compatibility with Tilings:** LBCLayer is very well suited for tiling based systems such as Human detector where individual tiles can be processed separately before being assembled.
- **Biological Inspiration:** Sparse and binary inspired filters mimic biological visual cortex processing that is helpful for edge detection and texture recognition.

Mathematical Summary

Considering the inputs x , the summary of the operation and gradients are as follows:

$$\text{Anchor weight: } A = f_{\text{binary}}(W, s)$$

$$x_1 = x * A$$

$$x_2 = \text{ReLU}(x_1)$$

$$x_{\text{out}} = x_2 * W_{1 \times 1}$$

Gradients: $\frac{\partial L}{\partial W_{1 \times 1}} \neq 0$, whereas $\frac{\partial L}{\partial A} = 0$. In this case, $*$ denotes a convolution, s denotes the sparsity threshold, and A denotes the fixed binary anchor weights.

Overview of Human Presence Detection (HPD)

The HPD (HumanDetector) is a **tile based convolutional network** designed to detect humans in images. Its key characteristics:

1. **Inputs:** $x \in \mathbb{R}^{N \times 1 \times H \times W}$ grayscale images.
2. **Tiling:** Divide image into `tile_rows` $\times$ `tile_cols` patches without overlaps.
3. **Per tile Feature Extraction:** The use of LBCLayer to perform sparse and task-aware feature extraction from each patch.
4. **Features Aggregation / Stitching:** Features from all the patches aggregated into one feature vector.
5. **Classifier Head:** The output dense layers that provide logit values for `num_classes`. (E.g., Human vs. Not Human).

Mathematically:

$$f_{\theta}(x) = \text{Classifier}(\text{Concat}_{i,j}(\text{TileExtractor}(x_{i,j}))) \quad (3-5)$$

Where $x_{i,j}$ is the (i, j) -th tile.

Tiling Stage

The tiling stage prepares the input for localized processing.

- **Input:** $x \in \mathbb{R}^{N \times 1 \times H \times W}$
- **Parameters:** `tile_rows`, `tile_cols`
- **Tile size:**

$$h = \frac{H}{\text{tile_rows}}, \quad w = \frac{W}{\text{tile_cols}} \quad (3-6)$$

- **Split input into tiles:**

$$x_{i,j} = x[:, :, i \cdot h : (i + 1) \cdot h, j \cdot w : (j + 1) \cdot w] \quad (3-7)$$

Purpose:

- Localizes feature extraction $\rightarrow$ each tile acts as a **node** (MCU-like processing).
- Improves **spatial granularity** for detecting humans.
- Tiles are **independent**, can be processed in parallel.

Tile Feature Extractor (LBC-based)

Every tile goes through *tile_extractor* whose structure is described below:

- **LBC Layer 1:** 1to16 channels with sparsity = 0.9.
- **MaxPool2d:** Kernel size 2 by reducing spatial dimensions by half.
- **LBC Layer 2:** 16to32 channels with sparsity = 0.9.
- **AdaptiveAvgPool2d:** pooling spatial dimensions to 1×1 .

For individual tile $x_{i,j}$:

$$x' = \text{LBCLayer}(x_{i,j}) = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x_{i,j}))) \quad (3-8)$$

Aggregation / Stitching

After feature extraction, the tile outputs get aggregated together:

1. **Flatten the output for each tile:** $f_{i,j} \in \mathbb{R}^{32}$
2. **Concatenate all tiles:** If number of tiles = $\text{tile_rows} \times \text{tile_cols}$:

$$f_{\text{combined}} = [f_{1,1}, f_{1,2}, \dots, f_{\text{tile_rows}, \text{tile_cols}}] \in \mathbb{R}^{32 \cdot \text{num_tiles}} \quad (3-9)$$

This process can be considered analogous to graph aggregation, treating every tile as a graph node

Classifier Head

The classifier performs an MLP on the concatenated vector:

- **Input:** $f_{\text{combined}} \in \mathbb{R}^{32 \cdot \text{number of tiles}}$
- **Layers:** $\text{Linear}(32 \cdot \text{number of tiles} \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64 \rightarrow \text{num_classes})$
- **Output Classification Logits:**

$$\hat{y} \in \mathbb{R}^{N \times \text{num_classes}} \quad (3-10)$$

Uses **CrossEntropyLoss** when training:

$$L = - \sum_{c=1}^{\text{num_classes}} y_c \log \text{softmax}(\hat{y}_c) \quad (3-11)$$

Computational Efficiency

The LBCLayer reduces Floating Point Operations (FLOPs) significantly:

$$\text{FLOPs} \approx C_{in} \cdot C_{out} \cdot k^2 \cdot H \cdot W \quad (3-12)$$

- **Binary conv:** Multiplications are basically simple additions/subtractions.
- **1x1 conv:** Provides lightweight mixing of channels.
- **Tile based processing:** Can be parallelized across multiple nodes.

Mathematical Summary of Forward Pass

1. Tile splitting:

$$\{x_{i,j}\}_{i,j=1}^{R,C} = \text{Tile}(x) \quad (3-13)$$

2. Feature extraction (per tile):

$$f_{i,j} = \text{AdaptiveAvgPool2d}(\text{LBCLayer2}(\text{MaxPool2d}(\text{LBCLayer1}(x_{i,j})))) \quad (3-14)$$

3. Flatten and concatenate:

$$f_{\text{combined}} = \text{Concat}([f_{i,j}]) \quad (3-15)$$

4. Classification:

$$\hat{y} = \text{Linear}_2(\text{ReLU}(\text{Linear}_1(f_{\text{combined}}))) \quad (3-16)$$

5. Loss:

$$L = - \sum_{c=1}^{\text{num_classes}} y_c \log \text{softmax}(\hat{y}_c) \quad (3-17)$$

Advantages of HPD Architecture

1. **Sparse and efficient:** LBC reduces compute and memory footprint.
2. **Tile based learning:** Learns local patterns for small human parts.
3. **Flexible:** Can adapt tile size and sparsity based on resources.
4. **Parallelization:** Processed independently per tile.
5. **Low data scenario:** Set anchors minimize training needs.

3.2. Hardware Requirements and system configuration

The design employs a distributed architecture that is multi-node based on the ESP32 and ESP32-S3 MCU. In order to maximize efficiency and take advantage of limited resources, the system is structured hierarchically in three levels.

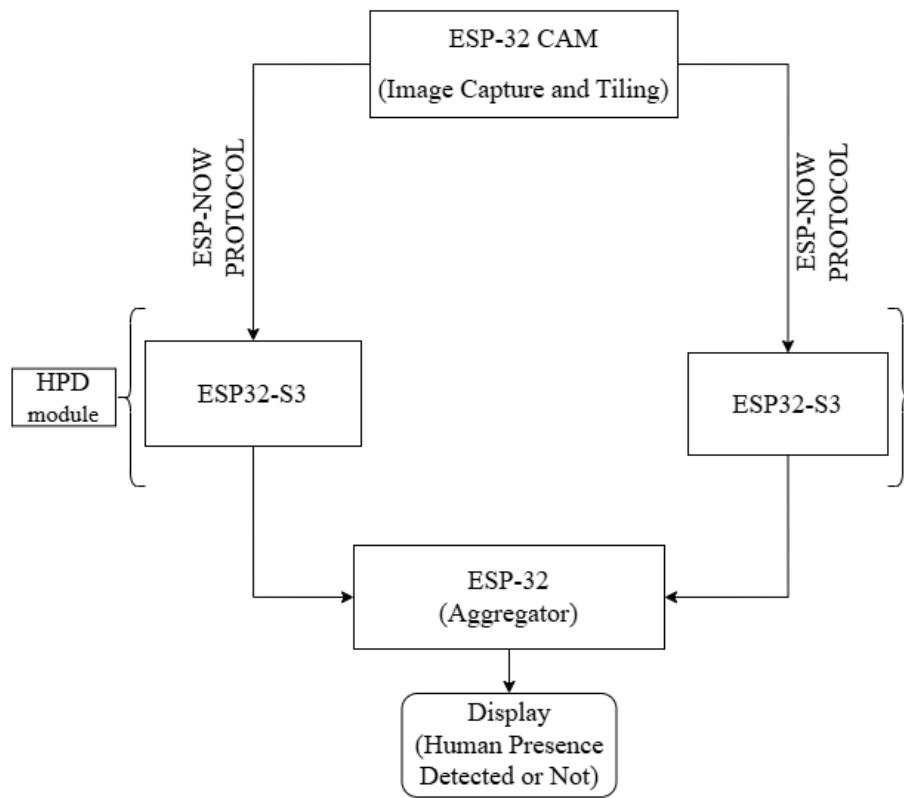


Figure 3-3. System Architecture showing main components and their interactions.

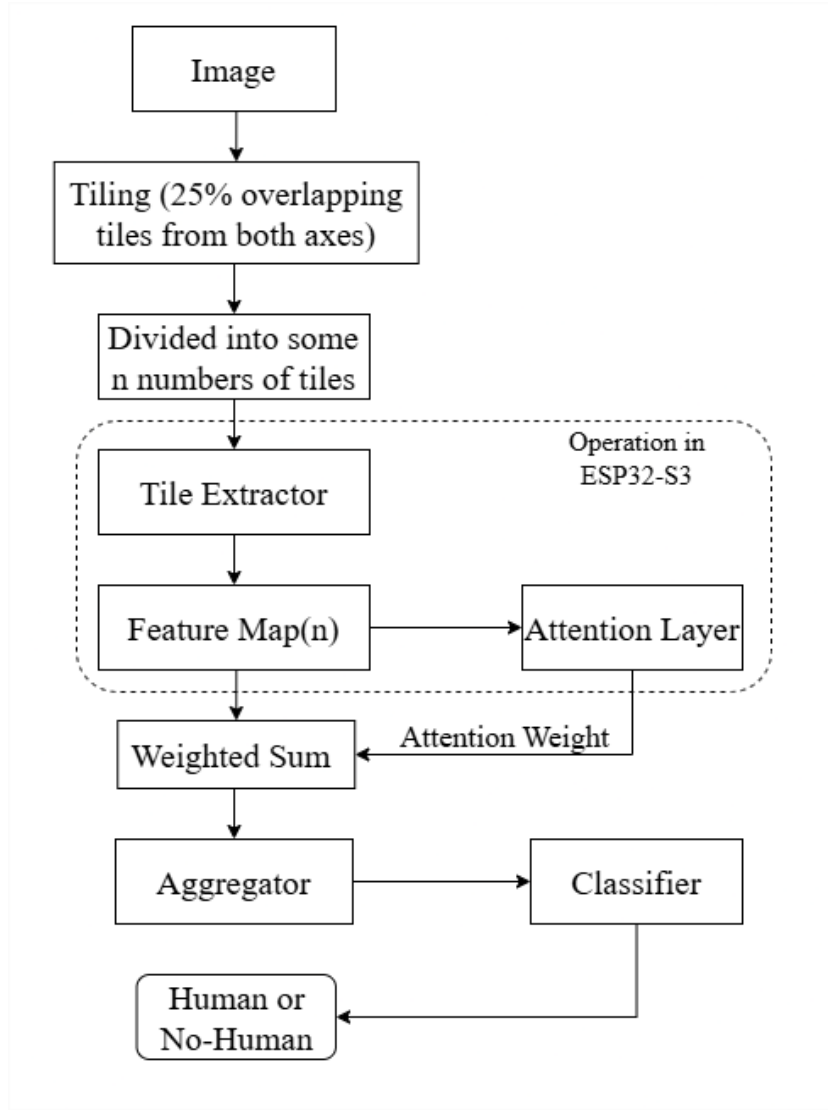


Figure 3-4. Detailed Data Flow Diagram of the Tiling and Inference Pipeline.

3.2.1. Input Layer

Centered around an ESP32-CAM [?], this layer manages high bandwidth data acquisition and initial pre processing. Its primary role is to prepare and distribute workloads to the processing tier through the following pipeline:

Luminance Conversion (Grayscale)

Processing three channel RGB data significantly inflates the memory buffer requirements and computational complexity ($O(3n)$ vs $O(n)$). The captured frame has to be converted to an 8 bit grayscale format using a weighted luminance formula as given in (3-18) [?].

$$P(y = 1 | x) = \frac{e^{z_{\text{person}}}}{e^{z_{\text{bg}}} + e^{z_{\text{person}}}} \quad (3-18)$$

Where z_{person} and z_{bg} represents the logits corresponding to *person* and *background* class, respectively. Data compression reduces the input data volume by 66.6%, hence leaving more room for heap memory allocation for tiling and transmission buffers.

$$Y = 0.299R + 0.587G + 0.114B \quad (3-19)$$

Partitioning Distribution

The process begins with the Tiling module, which extracts n overlapping tiles from the source image. This step is critical for maintaining high resolution features on resource constrained hardware. By using a 25% overlap between adjacent tiles, the system ensures that human subjects positioned at tile boundaries are not bisected, preserving spatial continuity for the feature extractor.

3.2.2. Communication Protocol Implementation

The Espressif Systems (ESP) protocol will be the communication backbone [?], selected for its highly efficient, peer to peer data exchange capabilities and low latency feedback. While ESP is efficient, its throughput is limited (observed maximums up to 771 Kilobyte (KB)/s). Therefore, the implementation will rely on efficient data packing, ensuring that tile data transfer maximizes the available throughput and that result packets adhere to the small packet size limits (< 250 bytes for control and aggregation data). The ability to transfer data quickly is paramount, as the latency gained by parallel processing can be quickly lost if the communication time dominates the total system latency.

3.2.3. Feature Extraction Layer

- **Parallel Processing:** Each node receives a specific low resolution tile from the Input Layer.
- **Local Inference:** Each node runs the HPD model on its assigned tile, extracting features and computing an attention weight that indicates the likelihood of human presence in that tile.
- **Output Generation:** Each node outputs a feature map and an attention weight, which are then transmitted to the Output Layer for aggregation and final decision-making.

3.2.4. Output Layer (Aggregation & Logic)

This layer is the last stage of the architecture and it is the core decision making layer. The feature map and the attention weight of each tile are taken from the feature extractor nodes, and the following operations are performed:

- **Attention score calculation:** The attention score for each tile is computed using the SoftMax function, which normalizes the quantized scores into a probability distribution. This allows the system to make weighted final decision.
- **Weighted feature aggregation:** Using the attention scores, we do a weighted sum of the feature maps from all tiles. This effectively combines the information, while giving more weight to tiles that are more likely to contain human features.
- **Final Classification:** The aggregated feature map is then passed to a classifier layer (e.g., fully connected layer) to compute a confidence score for the presence of a human in the whole scene. This score is then used to make a binary decision (human detected or not) that can be displayed on an output device such as an LCD screen.

3.3. Hardware Used

The core hardware of this project is based on the MCUs of the ESP32-family. It is chosen because they are low cost, low power, and can run edge-level machine learning tasks. This system uses a distributed architecture. Here, image capturing, processing and decision-making are separated across different ESP32 devices to overcome memory and processing limitations of a single microcontroller. This hardware design allows the system to perform real-time HPD without relying on cloud servers, making it faster, more private, and more energy efficient.

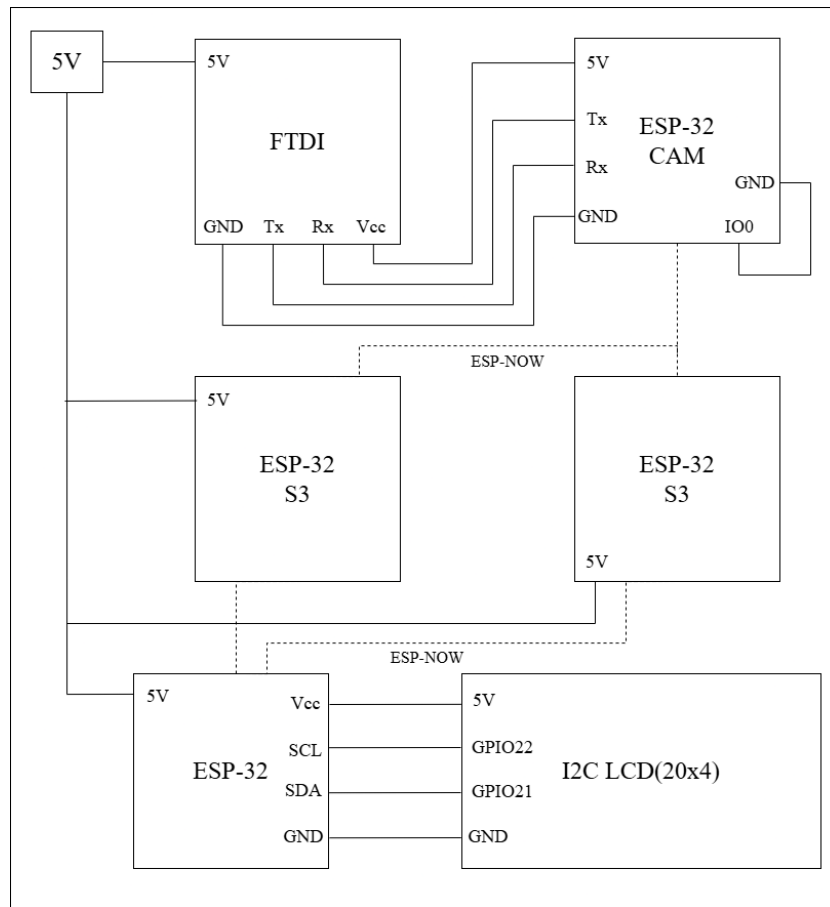


Figure 3-5. Wiring Interface between Programming (FTDI), Processing (ESP32-S3), and Visualization (I2C LCD) Units.

The hardware architecture as shown in 3-5 consists of a distributed processing network designed for real-time data acquisition and visualization. The system is partitioned into three primary functional blocks: the Sensing Node (ESP32-CAM), the Processing Cluster (ESP32-S3 units), and the Output Interface (I2C LCD connected via ESP32)

3.3.1. ESP32-CAM

The ESP-CAM module is used as the input hardware of the system. Its main role is to capture visual data from the environment. Here, the camera continuously captures images of 224×224 at a controlled rate of 1 every 15 seconds. Since color images require more memory and computation, the ESP-CAM converts the captured rgb images into grayscale format. After this, the high-resolution image is divided into smaller tiles of 96×96 with overlap of 25% between adjacent tiles. These smaller image tiles are easier to process and then transmit to other ESP devices using Espressif Non-Beaconing Wireless (ESP-NOW) protocol for further processing. Here, the tiles are evenly distributed between the two esp32s3 devices when the total number of tiles is even; if the number of tiles is odd, the first node is assigned one additional tile.

3.3.2. USB to TTL converter (FTDI)

The USB to TTL (FTDI) converter is used to connect the ESP32-CAM boards to a laptop for programming. It is because ESP32-CAM does not have a built-in USB interface for programming or serial communication. The FTDI module acts as a bridge between them since a laptop communicates using USB and the ESP32 uses TTL serial (UART) signals. In this project, the FTDI also supplies 5V power from the laptop via USB, which removes the need for an external power source.

Connect GPIO0 to GND in ESP32-CAM for flashing. This indicates that the ESP32-CAM is ready to receive new code. After uploading the code, we can disconnect GPIO0 from GND.

Table 3-1. Wiring the FTDI to ESP32-CAM

FTDI	ESP32-CAM
VCC	5V
GND	GND
RX	U0T
TX	U0R

3.3.3. ESP32-S3

The ESP32-S3 MCUs act as the processing or inference nodes in the system. Each ESP32-S3 receives a small image tile from the ESP32-CAM, on which it runs a Tiny Machine Learning (TinyML)-based HPD model. These boards are selected because they are optimized for AI workloads and support low-bit operations. These are necessary to run neural networks with limited memory. Each ESP32-S3 processes its assigned tile independently and decides if human features are present. Each ESP32-S3 receives a small image tile from the ESP32-CAM, on which it runs a Tiny Machine Learning

(TinyML)-based HPD model. These boards are selected because they are optimized for AI workloads and support low-bit operations. These are necessary to run neural networks with limited memory. Each ESP32-S3 processes its assigned tile independently and decides if human features are present. Instead of sending large image data back, it sends only a compact result, such as a confidence score and tile location, which minimizes communication overhead.

3.3.4. ESP32

The ESP32 board is used as the aggregation node of the system. It is used to collect the results from all ESP32-S3 and uses attention weight from all tiles to compute attention score using SoftMax function and then uses the attention score to perform weighted sum of feature maps of tiles. Then, it passes the new feature map through classifier layer to compute the confidence score. This confidence score is to form a final decision about human presence in the entire scene.

3.3.5. I2C LCD Display

In this system, the I2C LCD display is used to show the final output of the project. After the ESP32-S3 boards finish processing the image tiles and send their results, the ESP32 combines all the information and decides whether a human is present or not. This final decision is then shown on the LCD screen as **“Human Detected”** or **“No Human Detected”**. The I2C LCD has been chosen because it needs only two wires for communication. It keeps the wiring simple and saves GPIO pins on the ESP32.

Table 3-2. Wiring the I2C LCD to ESP32

I2C LCD Pin	ESP32 Pin	Purpose
VCC	3.3V or 5V	Power Supply
GND	GND	Ground Connection
SDA	GPIO21	Data Line
SCL	GPIO22	Clock Line

3.3.6. Power Supply

All hardware components in the system are powered through USB connections using a laptop. During development and testing, the laptop provides stable 5V power supply to the ESP32-CAM, ESP32-S3 and ESP32 boards.

3.3.7. Low Bit Quantization Implementation

The baseline operational model will use Post training Quantization (PTQ) to convert the HPD model to symmetric 8 bit Integer (INT8) weights and asymmetric INT8 activations. Further optimization will involve the experimental deployment of 4-bit

Integer (INT4) quantization for specific non critical layers. This mixed precision approach will push memory reduction to its architectural limits, targeting a final Flash footprint of < 1.0 Megabyte (MB). To compensate for the precision loss due to such aggressive compression, calibration datasets will be used along with layer wise equalization techniques. These methods balance weight ranges in the custom architecture with the aim of maximal accuracy retention less than 90% of the pruned 32-bit Floating Point 32-bit Floating Point (FP32) baseline. This optimization is necessary to provide a robust model for presence detection, and to fit within the 512 KB SRAM limit.

3.4. Evaluation Metrics and Performance Framework

3.4.1. Detection Reliability Matrix (The Deep Learning Layer)

The main goal is to fix the stationary detection issue that Passive Infrared (PIR) sensors have. The following standard computer vision criteria will be used to assess our CNN model:

Recall (Sensitivity)

Measured especially for participants who are motionless to demonstrate the system's capacity to sustain presence signals in situations where movement is minimal. The Equation is as shown in equation (3-3):

$$Recall = TruePositive(TP)/(TP + FalseNegative(FN)) \quad (3-20)$$

Precision

To ensure environmental noise (shadows, Heating, Ventilation, and Air Conditioning (HVAC) airflow) does not trigger false positives in the model's inference. The Equation is as shown in equation (3-4):

$$Precision = TP/(TP + FalsePositive(FP)) \quad (3-21)$$

3.4.2. Distributed System Efficiency Matrix (The Computing Layer)

We must measure the benefits of distributing the inference burden among several ESP32-S3 nodes using the Distributed Adaptive Tiling (DAT) framework as this project makes use of a Distributed Cluster.

End-to-End Latency (total): The duration between the Master node's picture acquisition and the ultimate detection outcome.

Memory Footprint: Maximum SRAM used by any single node during inference.

Communication Delay: The proportion of the entire cycle used for local Deep learning computation as opposed to ESP tile transmission.

4. REQUIREMENTS

The development of a distributed, tile-based HPD system requires a multi level analysis of hardware constraints, software dependencies, and network reliability. The section below discusses the basic requirements needed for the project to be feasible.

4.1. Hardware Requirements

Given the ESP32-S3 chip in the design, the choice of hardware is based on the speed up and power consumption needs related to Artificial Intelligence (AI).

- **Camera Module:** A high-resolution camera module(OV2640) capable of capturing images at higher resolution than traditional image processing systems, which will then be partitioned.
- **Input Node:** A master node responsible for taking images from the image sensor; partitioning them into multiple tiles and distributing them to edge nodes.
- **Edge Computing Nodes:** Multiple ESP32-S3 MCUs are required. These must support the *XtensaLX7* vector instructions to handle the tile-level CNN inference.
- **Aggregator Node:** A simple ESP32 responsible for collecting feature vectors and executing the final classification layer.
- **Communication Interface:** A low-latency communication system for inter-node communication ESP-NOWs.

4.2. Software & Framework Requirements

To achieve ultra-low bit quantization and distributed processing, the following software stack is required:

- **Development Environment:** Arduino IDE using C/C++.
- **AI Inference Library:** The Espressif Deep Learning library (ESP-DL) makes use of efficient kernels for quantization 8-bit or 16-bit computation.
- **Model Optimization:** Initial training and conversion of models with TensorFlow Lite for Microcontrollers (TFML) are followed by Quantization-Aware Training (QAT).
- **Tiling Logic:** A custom C++ pre-processing module to handle spatial partitioning and memory-safe buffer management for individual tiles.

4.3. Non-functional Requirements

- **Latency:** There will be low latency in order for this to be a real time application.
- **Scalability:** This would require an adjustable number of nodes, depending on the level of resolution and required Frames Per Second (FPS).

- **Robustness:** If one of the nodes is no longer working, it will need to fall back to the other nodes to get partial occupancy information.

5. EXPECTED OUTCOMES

A. APPENDICES

A.1. PROJECT BUDGET

Table A-1. Budget Estimation

S.N.	Hardware	Units	Cost (NPR)
1	ESP32-CAM	1	Rs. 1300
2	ESP32-S3	2	Rs. 3000
3	ESP32	1	Rs. 1000
4	FTDI	1	Rs. 350
5	Data Cables	4	Rs. 400
6	I2C LCD Display	1	Rs. 1000
7	Others	-	Rs. 1000
Grand Total			Rs. 8050

A.2. PROJECT TIMELINE

A.2.1. Gantt Chart

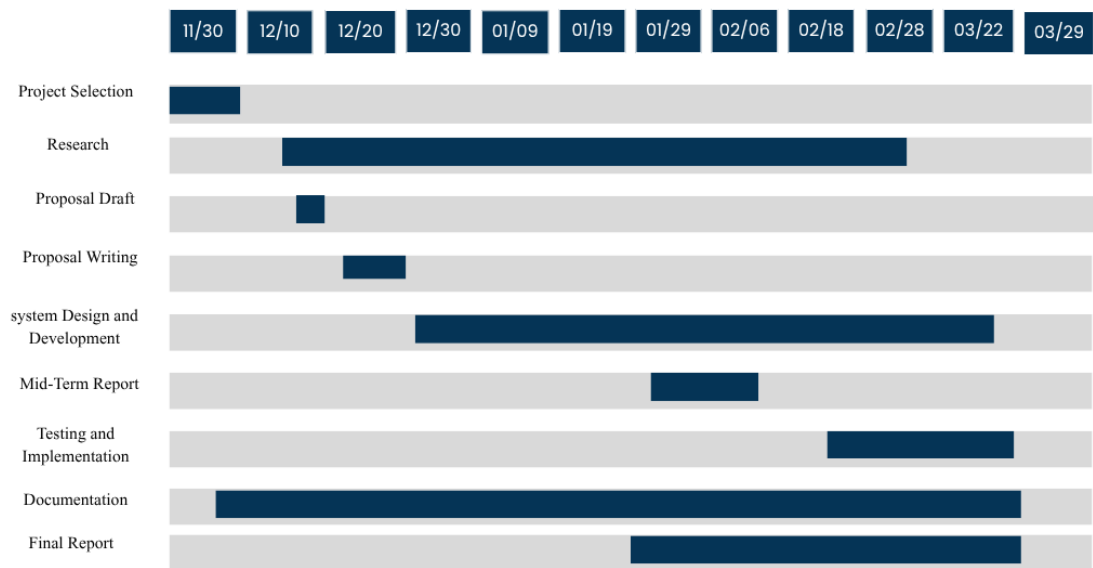


Table A-2. Gantt Chart for Project Timeline

REFERENCES

- [1] J. Kim, S. Park, and H. Lee, “Human pose estimation using MediaPipe pose and optimization method based on a humanoid model,” *Applied Sciences*, vol. 13, no. 4, p. 2700, 2023.
- [2] S. Sarkar, Y. Jang, and I. Jeong, “Multi-camera-based 3D human pose estimation for close-proximity human-robot collaboration in construction,” in *Proceedings of the 9th International Conference on Construction Engineering and Project Management (ICCEPM)*, Las Vegas, NV, USA, 2022.
- [3] P. Zell, B. Rosenhahn, and T. Müller, “Multimodal human motion dataset of 3D anatomical landmarks and pose keypoints,” *Data in Brief*, 2024.
- [4] S. Bultmann and S. Behnke, “Real-time multi-view 3D human pose estimation using semantic feedback to smart edge sensors,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [5] L. Bragagnolo, M. Terreran, D. Allegro, and S. Ghidoni, “Multi-view pose fusion for occlusion-aware 3D human pose estimation,” *arXiv preprint arXiv:2408.15810*, 2024.
- [6] V. Leroy, J.-S. Franco, and E. Boyer, “Probabilistic triangulation for uncalibrated multi-view 3D human pose estimation,” *arXiv preprint arXiv:2309.04756*, 2023.
- [7] K. Liang, Y. Chen, and Z. Wang, “Behavior imitation for manipulator control and grasping with deep reinforcement learning,” *arXiv preprint arXiv:2405.01284*, 2024.
- [8] X. Jiang, B. Wu, S. Li, Y. Zhu, G. Liang, Y. Yuan, Q. Li, and J. Zhang, “Multi-humanoid robot arm motion imitation and collaboration based on improved retargeting,” *Biomimetics*, vol. 10, no. 3, p. 190, 2025.
- [9] T. Faria, R. Sousa, and C. Santos, “Teleoperation system for service robots using a virtual reality headset and 3D pose estimation,” *Sensors*, 2025.
- [10] G. Moya-Alcover, A. Jaume-i Capó, and J. Varona, “Motion capture benchmark of real industrial tasks and traditional crafts for human movement analysis,” *IEEE Access*, 2023.